

CO2 AT1 – Linear Search: First Element Detection

Problem Statement

Given the unsorted dataset:

```
[  
[55,21,39,47,63,70]  
]
```

apply the **Linear Search algorithm** to find the required element.

Since the question states “**First Element Detection**”, we consider the target element as the **first element, 55**.

Linear Search checks elements sequentially from the beginning of the dataset until the target is found.

1. Objectives

The objectives are:

1. Understand the Linear Search technique.
2. Search for an element in an unsorted dataset.
3. Demonstrate sequential comparison of elements.
4. Identify the first occurrence of the target element.
5. Count the number of comparisons.
6. Analyze the time and space complexity.
7. Evaluate the efficiency of Linear Search.

2. Inputs and Outputs

Input

Dataset = [55, 21, 39, 47, 63, 70]

Target = 55

Output

Element 55 found at index 0.

Position = 1

Comparisons = 1

3. Assumptions and Constraints

Assumptions

- The dataset is unsorted.
- Searching begins from the first element.
- Indexing starts from 0.
- The search stops immediately when the target is found.
- The target is 55.

Constraints

- No sorting is performed.
- Elements are checked sequentially.
- Each element is compared with the target.
- The first matching occurrence is returned.

4. Problem Analysis

Linear Search is a simple searching technique that works on both **sorted and unsorted datasets**.

The algorithm starts from the first element and compares each element with the target.

For:

[55, 21, 39, 47, 63, 70]

and target:

55

the first comparison is:

```
[  
55 == 55  
]
```

The condition is true, so the search terminates immediately.

Therefore, no other elements need to be checked.

Search Trace

55 → 21 → 39 → 47 → 63 → 70

↑

Target found here

5. Mathematical Model

Let the dataset be:

[
A=[a₀,a₁,\ldots,a_{n-1}]
]

and let the target be (x).

Linear Search checks:

[
a_i=x
]

for:

[
i=0,1,2,\ldots,n-1
]

until a match is found.

For this problem:

[
A[0]=55
]

and:

[
x=55
]

Therefore:

[
A[0]=x
]

The search succeeds after **one comparison**.

6. Algorithm Selection

Selected Algorithm: Linear Search

Linear Search is appropriate because:

- The dataset is unsorted.
- No preprocessing or sorting is required.
- The target may occur at any position.
- The first element can be detected immediately.
- The algorithm is simple to implement.

Unlike Binary Search, Linear Search does **not require sorted data**.

7. Algorithm / Pseudocode

LINEAR_SEARCH(A, n, target)

for i = 0 to n-1

 if A[i] == target

 return i

return -1

For the given input:

A = [55, 21, 39, 47, 63, 70]

target = 55

Execution:

i = 0

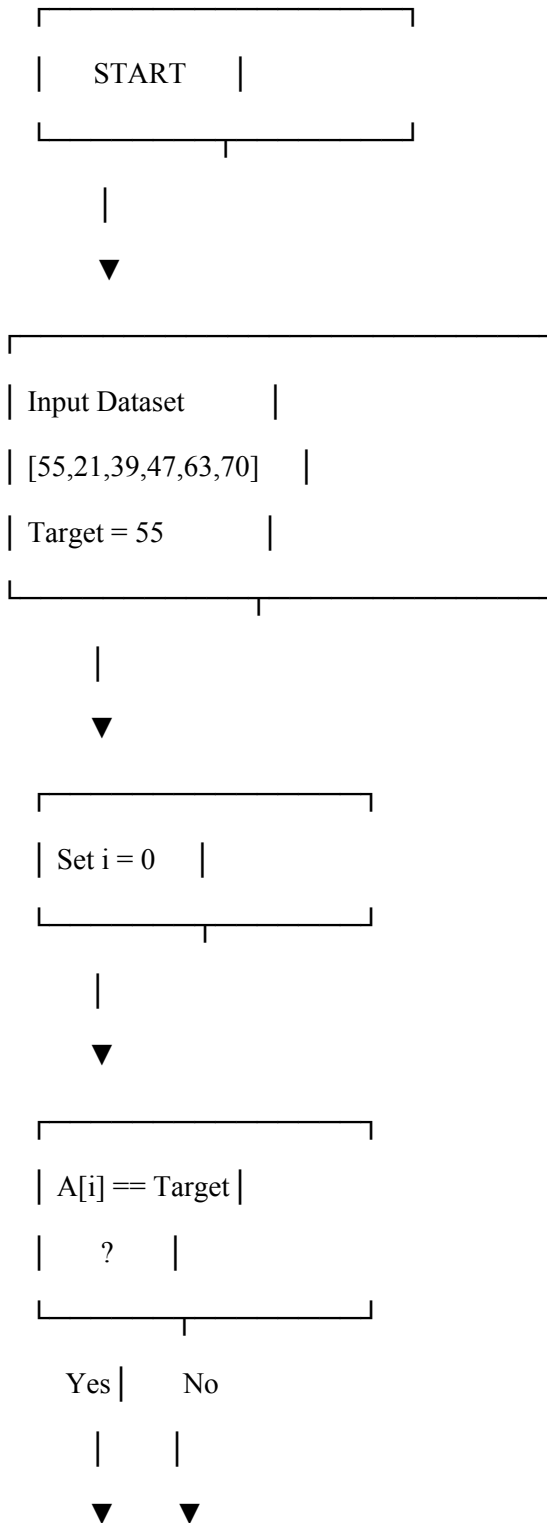
A[0] = 55

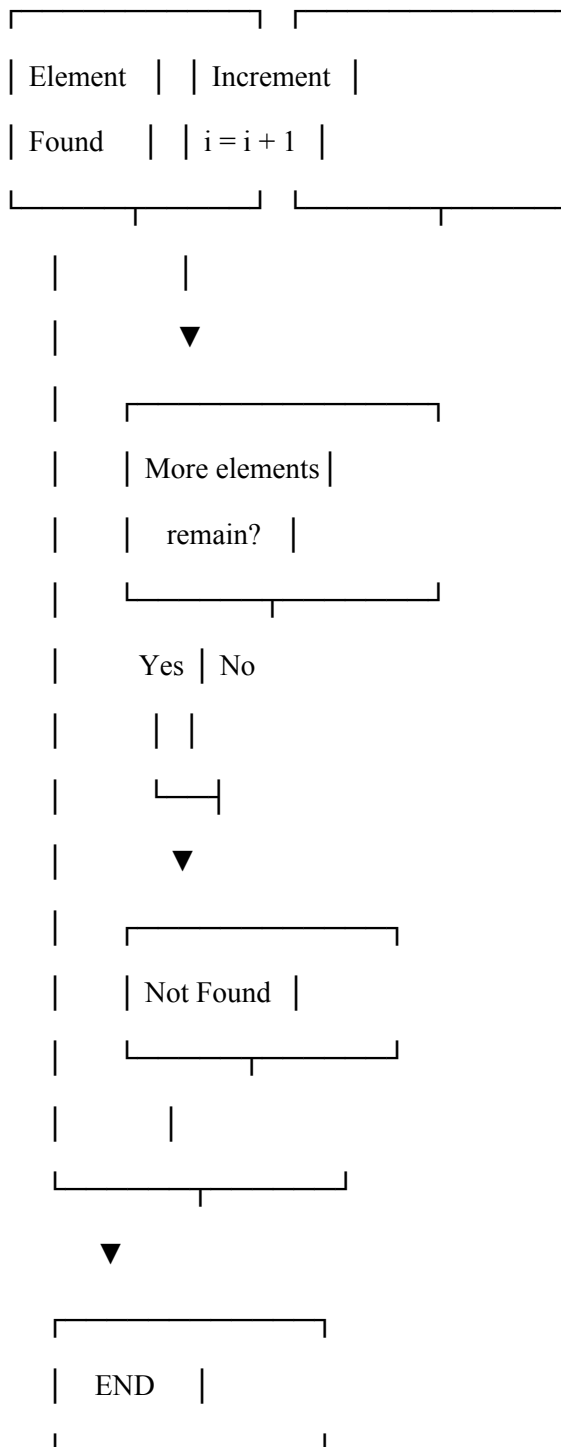
55 == 55

TRUE

Return 0

8. Flowchart





9. Step-by-Step Execution

Dataset:

[55, 21, 39, 47, 63, 70]

Target:

Step	Index	Element	Target	Comparison	Result
1	0	55	55	$55 == 55$	Match

Since the element is found at the first position, the algorithm immediately stops.

Final Search Result

```
[
\boxed{\text{Element } 55 \text{ found at index } 0}
]
```

Position using human-readable numbering:

```
[
\boxed{\text{Position}=1}
]
```

10. Number of Comparisons

Only one comparison is required:

```
[
55 == 55
]
```

Therefore:

```
[
\boxed{1 \text{ comparison}}
]
```

This represents the **best-case performance** of Linear Search.

11. Complexity Analysis

Best Case

The target is at the first position.

Number of comparisons:

```
[
1
]
```

Therefore:

[
\boxed{O(1)}
]

Average Case

If the target is equally likely to occur at any position, approximately half of the elements may need to be examined.

Therefore:

[
\boxed{O(n)}
]

Worst Case

The target is at the last position or does not exist.

All (n) elements must be examined.

Therefore:

[
\boxed{O(n)}
]

Space Complexity

Linear Search uses only a few variables such as:

i

target

No additional data structure is required.

Therefore:

[
\boxed{O(1)}
]

12. Correctness Analysis

The algorithm is correct because it checks every element sequentially until a matching element is found.

For the given dataset:

[55, 21, 39, 47, 63, 70]

the first element is:

```
[  
A[0]=55  
]
```

Since the target is also:

```
[  
55  
]
```

the condition:

```
[  
A[0]=target  
]
```

is true.

Therefore, the algorithm correctly returns index 0.

13. Test Cases

Test Case	Dataset	Target	Expected Result
1	[55,21,39,47,63,70]	55	Found at index 0
2	[55,21,39,47,63,70]	39	Found at index 2
3	[55,21,39,47,63,70]	70	Found at index 5
4	[55,21,39,47,63,70]	100	Not found
5	[55,55,21,39]	55	First occurrence at index 0

14. Results

Parameter	Result
Dataset	[55,21,39,47,63,70]
Dataset size	6
Target	55

Index found	0
Position	1
Comparisons	1
Search status	Found
Best-case time	$O(1)$
Average-case time	$O(n)$
Worst-case time	$O(n)$
Space complexity	$O(1)$

15. Performance Evaluation

The target element 55 is located at the **first position**. Hence, Linear Search performs extremely well for this particular input.

Only:

```
[
\boxed{1}
]
```

comparison is required instead of checking all six elements.

However, this performance should not be generalized to every search operation.

For example, searching for 70 would require:

$55 \rightarrow 21 \rightarrow 39 \rightarrow 47 \rightarrow 63 \rightarrow 70$

resulting in **6 comparisons**.

Searching for an element that does not exist would also require checking all six elements.

Therefore, Linear Search has:

- **Best case:** $O(1)$
- **Average case:** $O(n)$
- **Worst case:** $O(n)$

16. Advantages

1. Simple and easy to implement.
2. Works with unsorted data.
3. Requires no preprocessing.
4. Uses constant auxiliary space.
5. Efficient when the target is near the beginning.
6. Suitable for small datasets.
7. Can detect the first occurrence easily.

17. Limitations

1. Slow for large datasets in the average and worst cases.
2. May require checking every element.
3. Has $O(n)$ average and worst-case time complexity.
4. Does not take advantage of sorted data.
5. Less efficient than Binary Search for large sorted datasets.

18. Comparison with Binary Search

Feature	Linear Search	Binary Search
Data requirement	Unsorted or sorted	Must be sorted
Best case	$O(1)$	$O(1)$
Average case	$O(n)$	$O(\log n)$
Worst case	$O(n)$	$O(\log n)$
Implementation	Simple	More complex
Suitable for unsorted data	Yes	No
Preprocessing required	No	Sorting may be required

For the given unsorted dataset, Linear Search is the appropriate choice because sorting the data first would introduce additional processing.

19. Conclusion

The Linear Search algorithm was successfully applied to the unsorted dataset:

```
[  
  \boxed{[55,21,39,47,63,70]}  
]
```

to locate the target element:

```
[  
  \boxed{55}  
]
```

The algorithm begins at the first element and compares it with the target. Since:

```
[  
  55=55  
]
```

the target is found immediately at **index 0**, or **position 1**.

Only **one comparison** is required, representing the best-case performance of Linear Search.

The experiment demonstrates that Linear Search is simple, requires **$O(1)$ auxiliary space**, and works effectively with unsorted datasets. However, its average and worst-case time complexity is **$O(n)$** , making it less suitable for large datasets. For large **sorted** datasets, Binary Search provides better performance with **$O(\log n)$** time complexity.